AI Compiler Performance Engineering

Auto-Loops Books — ch22

Contents

1 Compiler-Driven End-to-End Performance Tuning **1**

1.1 HW aware profiling 3

1.2 bottleneck locating 8

1.3 tuning workflow 13

1.4 regression gates 17

1.5 Key Takeaways 22

1.6 Conclusion 24

Bibliography **25**

Chapter 1

Compiler-Driven End-to-End Performance Tuning

A decode compiler team can win every microbenchmark and still lose the fleet when nobody owns the *end-to-end* tuning loop—profile, locate, change, gate, repeat—under the bytes/token and launches/token counters Chapter 1 made non-negotiable (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). Part VI gave you backends; Chapter 21 gave you benchmark harnesses; this chapter wires them into a **compiler-driven E2E workflow** that treats each compiler knob as a hypothesis until decode telemetry confirms it (Chen, 2020; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259× on H100 by removing host overhead—yet cannot fix illegal HBM traffic when fusion boundaries still export activations (Chen, 2026; NVIDIA, 2024). E2E tuning exists to catch that gap: the moment a compiler change improves a GEMM kernel but raises bytes/token, the workflow rejects it before merge (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

The loop has four stations—hardware-aware profiling, bottleneck locating, structured tuning workflow, regression gates—each bound to multi-hardware residency rules from Chapter 2 (Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024). Fregly's profile-first culture applies throughout: Nsight and roofline charts justify knob changes more than intuition about tile sizes (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026). Chapter 25 extends this loop with automated search; Chapter 26 packages the artifacts for production (Chen and YiRage Contributors,

2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Industrial teams document the loop as a runbook, not tribal knowledge. New hires should reproduce a baseline profile on day one, classify a bottleneck on day two, and propose a single-knob experiment by day three—all against the same decode cell (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). When the runbook is missing, compiler teams revert to GEMM microbench heroics (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Ownership matters: profiling belongs to whoever changes IR; gates belong to release engineering; classification belongs to a cross-functional review with serving SREs present (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler-only tuning loops ship illegal schedules that frameworks mask until load rises (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Anti-patterns to post on the wall: (1) tune tiles while launches/token exceed kernel time; (2) merge compiler wins without canary; (3) profile prefill, ship decode; (4) skip IR diff review; (5) treat framework batching wins as compiler fusion wins (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The sections below replace each anti-pattern with a concrete practice (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

E2E tuning also bounds *when to stop*: if bytes/token and launches/token meet SLO after graph replay, MegaKernel investment waits—mode selection from Chapter 12 applies before endless fusion (NVIDIA, 2024; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016;

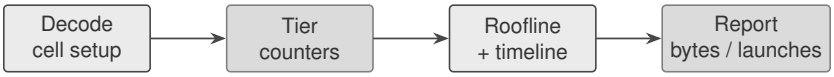


Figure 1.1: Hardware-aware profiling pipeline: frozen decode cell → residency counters → roofline/timeline → bytes/token and launches/token report.

Counter	**GPU tool surface**	**CPU / NPU delta**
bytes/token	Nsight memory chart, CUPTI	perf/LBR, DMA byte counts
launches/token	CUDA API trace, CUPTI	thread launches, sync points
ms/token	end-to-end decode cell	same cell, power cap noted
legality	graph capture, shared mem	static schedule, tile slots

Table 1.1: Hardware-aware profiling maps the same decode questions to different toolchains.

SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Good enough under measured decode goodput beats perfect under imaginary FLOPs (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

1.1 HW aware profiling

Hardware-aware profiling fails when teams profile prefill shapes, batch-32 training graphs, or synthetic GEMMs while production runs BS=1 decode with paged KV (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The fix is a *frozen decode cell*: pinned model revision, context bucket, sampling flags, warmup protocol, and SKU—logged beside every trace so comparisons are apples-to-apples (Chen, 2026, 2020; Chen and YiRage Contributors, 2026).

GPU profiling binds to shared-memory occupancy, TMA transactions, and global bytes moved per decode step—not SM utilization alone (NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025). CPU profiling binds to LLC misses, NUMA cross-socket traffic, and OpenMP gang efficiency (Goodfellow *et al.*, 2016; Chen, 2020). NPU/XDNA profiling counts DMA edges and static tile residency; many GPU counters simply do not exist—substitute compile-time buffer accounting plus on-device cycle estimates (AMD, 2024b,a; Chen and YiRage Contributors, 2026).

Translate profiler output into Chapter 1 vocabulary before opening a compiler

ticket: if bytes/token is within 10% of analytic HBM roofline (R_{floor}), orchestration tweaks (graphs, fusion) may still help; if bytes dominate, tile changes without fusion are theater (Chen, 2026; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). YiRage and similar stacks export IR-level residency metadata so profiles can be tied to specific fusion boundaries rather than mystery kernels (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023).

Warmup discipline separates steady-state decode from cold-start stories: graph capture, allocator priming, and CUDA context creation belong in the profile protocol document—not in ad-hoc Slack threads (NVIDIA, 2024; Chen, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Teams that compare profiles without matched warmup compare different phases and waste compiler cycles (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Tooling hygiene: store raw '.nsys-rep', CUPTI CSV, or NPU trace bundles beside summarized bytes/token tables in artifact storage keyed by git SHA (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Summaries alone rot; raw traces enable re-analysis when classification rules improve (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Prefill vs decode profiling split: compiler teams often inherit prefill-heavy traces from training infra (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). Mandate decode-cell scripts in CI and reject profile uploads that omit BS=1 metadata (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hardware-aware profiling is a contract, not a one-off Nsight session (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a;

Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware delta. Hopper profiles expose TMA and warp-specialization; XDNA profiles expose tile SRAM fill; CPU profiles expose cache hierarchy—identical decode step, incomparable dashboards unless normalized to bytes/token (SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016).

Example 1.1.1. Decode cell (BS=1, ctx=2048): a profile showing low SM occupancy but high HBM throughput is bandwidth-bound—compiler tuning should target fusion and KV residency, not bigger GEMM tiles (Chen, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022).

1.1.1 Defining the decode cell protocol

The decode cell protocol is the first artifact of the profiling station, and it must be precise enough that two engineers on two continents reproduce the same trace (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). The protocol records the model revision, tokenizer, weight format, context buckets, sampling flags, page-table layout, warm-up steps, and the exact measurement window (Kwon *et al.*, 2023; Chen, 2020). It also records what is excluded: cold-start costs, allocator priming, driver compilation, and graph capture belong in separate phases with separate labels (NVIDIA, 2024; Chen, 2026). A cell that omits any of these fields produces numbers that look comparable and are not (Davies *et al.*, 2025; Chen, 2020). The protocol should be checked into the repository with a schema validator so every uploaded profile either conforms or is rejected (Chen, 2026; Vellaisamy *et al.*, 2026).

1.1.2 Profiler overhead discipline

Profiling changes what it measures: capture buffers, symbolization, and per-kernel callbacks add overhead that shifts timings and can alter scheduling (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026). The protocol should therefore measure each workload twice—once with lightweight counters and once with full tracing—and report both so the overhead is visible (Chen, 2020; Vellaisamy *et al.*, 2026). Lightweight counters establish the steady-state baseline; full traces explain the baseline's composition (Chen, 2026; Davies *et al.*, 2025). When profiling overhead exceeds an agreed fraction of total runtime, the profile is a different workload and should be labeled as such (Vellaisamy *et al.*, 2026; Chen, 2020).

Teams that skip this discipline optimize artifacts they never actually run (Chen, 2026; Davies *et al.*, 2025).

1.1.3 Raw trace storage and re-analysis

Summarized tables answer today's question, but raw traces answer next quarter's question, which is why both must be stored (Davies *et al.*, 2025; Chen, 2020, 2026). Each raw trace bundle should carry the git SHA, the decode-cell protocol version, the tool version, and the machine fingerprint (Vellaisamy *et al.*, 2026; Chen, 2026). Storage cost is real, so the policy should keep full traces for the shipped cell matrix and retain summaries plus derived counters for exploratory runs (Chen, 2020; Davies *et al.*, 2025). When classification rules improve, re-analysis of old traces can explain historical regressions that summaries cannot (Chen, 2026; Davies *et al.*, 2025). Raw-trace retention is the difference between a profiling culture and a profiling archive (Vellaisamy *et al.*, 2026; Chen, 2026).

1.1.4 Normalizing counters across toolchains

GPU profilers, CPU perf counters, and NPU traces speak different languages, so the report layer must normalize them to the same three counters before comparison (Davies *et al.*, 2025; AMD, 2024b; Goodfellow *et al.*, 2016). Bytes moved per token and launches or dispatches per token are the portable vocabulary; raw tool metrics such as SM occupancy or DMA edge count supplement rather than replace them (Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016). The normalization rules belong in the protocol, including how DMA edge counts map to the launch concept and how cache hits count as bytes (Chen, 2020; Vellaisamy *et al.*, 2026). A normalized report lets a multi-hardware team compare a GPU regression to an NPU regression without translating profiler dialects by hand (Davies *et al.*, 2025; Chen, 2026). Without normalization, every cross-target discussion starts from incompatible dashboards (AMD, 2024b; Goodfellow *et al.*, 2016).

1.1.5 Runbook onboarding exercises

The tuning runbook becomes real when new engineers complete three exercises against it (Chen, 2020, 2026; Vellaisamy *et al.*, 2026). Day one is reproducing the baseline profile from the frozen decode cell and confirming the counters match the stored report (Davies *et al.*, 2025; Chen, 2026). Day two is classifying a synthetic bottleneck injected into the cell—for example an extra unfused activation—and writing the evidence-based label (Chen, 2020; Davies *et al.*, 2025). Day three is

filling a hypothesis card, running the experiment, and publishing the verdict (Chen, 2026; Vellaisamy *et al.*, 2026). Each exercise checks that the workflow artifacts exist and that the process is teachable rather than tribal (Chen, 2020; Davies *et al.*, 2025). A runbook that cannot onboard an engineer in three days is documentation, not a workflow (Chen, 2026; Vellaisamy *et al.*, 2026).

1.1.6 Environmental drift capture

Profiles change when the environment changes, even when the model does not (Chen, 2020, 2026; Davies *et al.*, 2025). Driver versions, CUDA libraries, allocator behavior, machine co-tenancy, and thermals all shift counters, so every report should record the environment fingerprint and flag out-of-band runs (Vellaisamy *et al.*, 2026; Chen, 2026). The fingerprint includes the driver, CUDA, library versions, host model, and the machine's measured baseline on a calibration kernel (Chen, 2020; Davies *et al.*, 2025). When a report's environment differs from the reference cell, the comparison is invalid and should be labeled as such rather than silently combined (Chen, 2026; Vellaisamy *et al.*, 2026). Environmental drift capture is what keeps a three-month-old baseline meaningful (Davies *et al.*, 2025; Chen, 2026).

1.1.7 Cold-start and steady-state labeling

Decode products experience both cold-start and steady-state phases, and the two must never share a counter row (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Cold start includes model load, graph capture, allocator priming, and first-token compile costs; steady state measures the repeated per-token path after warm-up (NVIDIA, 2024; Chen, 2026). The protocol should label every row by phase so an optimization that improves steady state but worsens cold start is visible as a trade rather than a regression or a win (Chen, 2020; Davies *et al.*, 2025). Products whose SLO binds cold start need a different tuning target than products that bind steady state (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Phase labeling is the smallest change that prevents most profile-comparison mistakes (Chen, 2026; Davies *et al.*, 2025).

1.1.8 Profiling skills for compiler engineers

Compiler engineers need profiling literacy because their changes are validated in profiles, not in unit tests (Chen, 2020; Davies *et al.*, 2025; Chen, 2026). The team should run a shared training curriculum covering the decode cell, the profiler

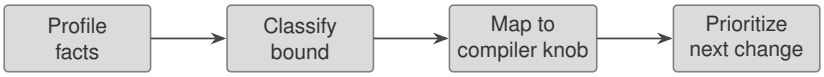


Figure 1.2: Bottleneck locating: profile facts → launch vs memory vs residency class → compiler knob → prioritized experiment.

toolchain per target, the counter normalization rules, and the classification tree (Vellaisamy *et al.*, 2026; Chen, 2026). Every engineer should be able to produce a labeled profile and defend the label with evidence before changing a compiler pass (Chen, 2020; Davies *et al.*, 2025). Cross-training between compiler and serving teams matters because the boundary tickets require both vocabularies (Kwon *et al.*, 2023; Chen, 2026). Profiling literacy is what prevents the workflow from becoming a ceremony performed only by a few specialists (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

1.1.9 Normalization quality checks

Counter normalization can itself introduce errors, so the workflow should include quality checks that validate the normalized numbers against a hand-verified reference trace (Chen, 2020, 2026; Davies *et al.*, 2025). The checks compare the normalized bytes/token and launches/token against the raw profiler output for a small reference workload and alert on unexplained divergence (Vellaisamy *et al.*, 2026; Chen, 2026). They also verify that toolchain version changes do not alter the mapping rules silently (Chen, 2020; Davies *et al.*, 2025). A normalization bug can mislabel every workload in the fleet, so the check runs before the nightly matrix rather than after a suspicious report appears (Chen, 2026; Vellaisamy *et al.*, 2026). Normalization quality checks are the metadata hygiene that keeps cross-target comparisons trustworthy (Davies *et al.*, 2025; Chen, 2026). They should also run on every profiling-toolchain upgrade before new traces are trusted, because a changed counter definition is as dangerous as a changed workload (Davies *et al.*, 2025; Chen, 2026).

1.2 bottleneck locating

Bottleneck locating is where E2E tuning separates senior compiler engineers from kernel tourists (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2020). The production surprise is not peak FLOPs—it is misclassification: teams fuse operators while launches/token remain launch-bound, or capture CUDA Graphs

while bytes/token stay at eager levels (Chen, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Use a decision tree grounded in measured counters:

1. **Launch-bound:** high launches/token, low R_{floor} proximity → graph mode, op fusion, MegaKernel seams (Chapter 12) (NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

2. **Memory-bound:** bytes/token near roofline → fusion depth, KV layout, attention kernel choice (Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; Chen, 2026).

3. **Residency-illegal:** correct counters but p99 spikes → spill paths, graph illegality, dynamic alloc in capture region (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

4. **Framework-bound:** compiler clean but Python/scheduler overhead → hand off to serving stack (Chapter 28), do not keep retuning tiles (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Diff lowered IR or CUDA when classification is unclear: new global allocations between attention and MLP usually explain bytes/token regressions better than guesswork (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). On NPU, diff static DMA schedules; on CPU, diff cache-blocking parameters in oneDNN/TorchInductor output (AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020).

MoE and speculative decode add **routing-bound** buckets: expert imbalance shows up as latency tails even when per-expert kernels look healthy (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Locate whether the bottleneck is compile-time expert fusion, runtime routing, or both before changing schedules (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Attention-backend swaps (FlashAttention, SDPA, vendor libs) can move bytes/token without changing launches/token—classify whether the bottleneck lives in attention or MLP/residual before celebrating attention wins (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). TaxBreak-style launch counts decompose where framework fragmentation

still dominates (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022).

Cross-SKU classification differs: a launch-bound Hopper cell may be memory-bound on a bandwidth-starved SKU at the same model (Chen, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Bottleneck labels are properties of (model, bucket, SKU), not universal truths (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

When profiles disagree with production dashboards, suspect bucket mismatch (context length), batching policy, or speculative path divergence—reconcile framework counters with compiler-cell counters before filing compiler bugs (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Bottleneck locating is forensic work (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Review gate. Every bottleneck ticket attaches profile artifacts, classification label, and the single compiler knob under test—no unfocused "make it faster" requests (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026, 2020).

1.2.1 Evidence rules for classification

Bottleneck labels must be earned with evidence, not asserted (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). A launch-bound label requires the launch count and kernel-time comparison; a memory-bound label requires bytes/token against the analytic roofline; a residency label requires the buffer accounting and spill evidence; a framework label requires the host-side timeline (NVIDIA, 2024; Dao *et al.*, 2022; Chen, 2020). Each label should cite the counter row that supports it and the counter row that rules out the alternatives (Chen, 2026; Davies

et al., 2025). When two labels are plausible, the ticket should say so and list the discriminating experiment (Chen, 2020; Vellaisamy *et al.*, 2026). Evidence rules are what keep bottleneck meetings from becoming opinion contests (Davies *et al.*, 2025; Chen, 2026).

1.2.2 Compiler-boundary and framework-boundary tickets

The most expensive classification error is opening a compiler ticket for a framework problem or vice versa (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). If the compiled kernels are clean but the timeline shows Python dispatch, allocator churn, or scheduler gaps between kernels, the bottleneck is upstream of the compiler and belongs to the serving stack (Kwon *et al.*, 2023; Chen, 2026). If the host timeline is clean and the device timeline shows spills or extra launches, the bottleneck is inside the compiled artifact (Chen, 2020, 2026). The boundary decision should be recorded on the ticket with the evidence for both sides, so the right owner picks it up immediately (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Boundary tickets also need an escalation path when the two teams disagree about who owns the gap (Chen, 2020; Davies *et al.*, 2025).

1.2.3 Multi-SKU classification matrix

Bottleneck labels are not portable across SKUs, so the team should maintain a classification matrix with one row per device class and context bucket (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). The same model can be launch-bound on a high-bandwidth GPU, memory-bound on a smaller SKU, and framework-bound on a CPU deployment (NVIDIA, 2022; Goodfellow *et al.*, 2016; Chen, 2026). Each matrix cell records the classification evidence and the last date it was measured (Chen, 2020; Davies *et al.*, 2025). When a fleet adds a new SKU, the first task is to fill its matrix cells before tuning anything (Chen, 2026; Vellaisamy *et al.*, 2026). The matrix prevents the classic mistake of promoting an optimization that was correct for the measured SKU to an unmeasured one (Davies *et al.*, 2025; Chen, 2026).

1.2.4 Worked classification walkthrough

Walk one dense decoder cell through the tree to make the method concrete. The profile shows hundreds of launches per token with low bytes/token relative to the roofline, so the first label is launch-bound (Vellaisamy *et al.*, 2026; Chen, 2026). The discriminating experiment captures the launch sequence and re-measures: if

ms/token drops toward the memory floor, the label is confirmed and the fix is graph capture plus fusion of the remaining seams (NVIDIA, 2024; Chen, 2026). If bytes/token rises during capture, the second label appears—residency-illegal—and the fix moves upstream to fusion legality before launch collapse (Dao *et al.*, 2022; Chen, 2026). The walkthrough is the template every ticket should follow: one profile, one primary label, one discriminating experiment, one knob (Davies *et al.*, 2025; Chen, 2020). Repeating the walkthrough weekly builds the pattern recognition that makes classification fast (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

1.2.5 Stop conditions and SLO boundaries

End-to-end tuning needs explicit stop conditions so effort does not continue past the point of diminishing returns (Chen, 2020, 2026; Vellaisamy *et al.*, 2026). When the measured bytes/token and launches/token meet the product SLO at the shipped buckets, the loop pauses and the remaining budget moves to the next bottleneck or the next tier (Davies *et al.*, 2025; Chen, 2026). Stop conditions must be written in terms of counters and SLOs, not in terms of compiler aesthetics, because the temptation is always to fuse one more operator (Chen, 2020; Vellaisamy *et al.*, 2026). The boundary should also define what is out of scope: if the residual gap is framework scheduling, the ticket leaves the tuning loop and enters the serving stack (Kwon *et al.*, 2023; Davies *et al.*, 2025). A workflow without stop conditions is a treadmill, not a system (Chen, 2026; Vellaisamy *et al.*, 2026).

1.2.6 Profiling budget ownership

Profiling consumes machine time and engineer attention, so it needs a budget and an owner just like the experiment queue (Chen, 2020, 2026; Davies *et al.*, 2025). The budget covers the nightly matrix, the canary tiers, and the exploratory runs attached to hypothesis cards (Vellaisamy *et al.*, 2026; Chen, 2026). An owner reviews which profiles are being collected, which are being re-analyzed, and which have become redundant (Chen, 2020; Davies *et al.*, 2025). Profile collection without analysis is waste that makes the archive harder to search (Chen, 2026; Vellaisamy *et al.*, 2026). Budget ownership keeps the profiling station sustainable rather than letting it grow without bound (Davies *et al.*, 2025; Chen, 2026).

1.3 tuning workflow

Tuning workflow turns classifications into a repeatable experiment queue (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Industrial teams use a kanban-style pipeline: *hypothesis* (knob + expected counter movement) → *branch build* → *decode-cell A/B* → *review* → *merge or discard* (Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

One change per experiment: fusion depth *or* tile size *or* backend switch—never three at once, or postmortems cannot attribute wins (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Keep a *human prior* schedule checked in as baseline; candidates must beat prior on bytes/token, launches/token, and ms/token simultaneously on the target SKU (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Chen and YiRage Contributors, 2026; Chen, 2020).

Workflow stages map to Part VI backends: XLA flags, TVM schedules, Triton configs, MLIR pass pipelines—each logged with version hashes (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; AMD, 2024b). Cross-backend experiments require separate hypothesis cards; promoting a Triton win to an XLA fleet without re-profile repeats the GPU-only culture Chapter 1 rejected (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b).

Canary deployment belongs in the workflow: 5–10% of decode replicas run candidate schedules with automatic rollback when p99 or error rate breaches SLO (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler tuning without canary is how clusters learn about illegal graph capture at 2am (NVIDIA, 2024; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2022).

Experiment queues should cap WIP: three concurrent knob experiments per layer block maximum, or attribution collapses and profiles diverge (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Document expected counter deltas on each hypothesis card—"↓ launches/token" vs "↓ bytes/token"—so reviewers reject mismatched outcomes quickly (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Rollback paths must be rehearsed: fingerprinted human-prior schedules restore in one deploy step when canary breaches SLO (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Tuning workflow is incomplete without a tested undo button (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware tuning runs parallel queues per fleet tier—never merge GPU and NPU experiment conclusions into one narrative (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Shared methodology, separate queues, separate promotion gates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a; Goodfellow *et al.*, 2016; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 1.3.1. Hypothesis card: "Increase MLP fusion depth by one operator; expect ↓ launches/token, ⇓ bytes/token if residency legal; measure ctx$\in \{512, 2048, 8192\}$." Reject if any bucket regresses (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Kwon *et al.*, 2023).

1.3.1 Hypothesis card schema

Every experiment starts as a hypothesis card with a fixed schema: the knob, the expected counter movement, the mechanism, the target cells, and the acceptance rule (Chen, 2020; Davies *et al.*, 2025; Chen, 2026). The knob must be one variable, named with its exact flag or schedule; the expected movement must name one or more counters and predict the direction (Vellaisamy *et al.*, 2026; Chen, 2026). The mechanism sentence explains why the change should move that counter, which is what lets reviewers reject cards whose mechanism contradicts the classification

evidence (Davies *et al.*, 2025; Chen, 2020). The acceptance rule states the bucket set and the tolerance, so the experiment has a predefined verdict rather than a negotiated one (Chen, 2026; Vellaisamy *et al.*, 2026). Cards with vague knobs or vague expectations should be returned before any build starts (Chen, 2020; Davies *et al.*, 2025).

1.3.2 Experiment queue operations

The tuning queue needs the same operational discipline as a code queue: owners, WIP limits, priority by expected impact, and a review gate for every merge or discard (Chen, 2020, 2026; Davies *et al.*, 2025). Priority should come from the classification matrix—fix the bottleneck that costs the most ms/token first—rather than from whoever shouts loudest (Vellaisamy *et al.*, 2026; Chen, 2026). Each completed experiment should publish its verdict even when the change is discarded, because negative results prevent other engineers from repeating the same dead end (Chen, 2020; Davies *et al.*, 2025). Queue metrics—cycle time, merge rate, discard reasons—should be reviewed monthly to find where the workflow itself is slow (Chen, 2026; Vellaisamy *et al.*, 2026). A tuning queue without operational metrics is a list, not a system (Davies *et al.*, 2025; Chen, 2026).

1.3.3 Canary design specifics

Canary design must specify the traffic split, the decision window, and the rollback condition before the candidate ships (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The canary should compare p99 and error rate against the baseline tier under the same request mix, with a predefined breach threshold (Chen, 2026; Davies *et al.*, 2025). Compiler candidates also need artifact-level canaries: confirm the shipped binary is the benchmarked binary by matching fingerprints at load time (Chen, 2020, 2026). A canary that runs for days without a decision is a failed process, so the window should be bounded by traffic volume rather than calendar time (Kwon *et al.*, 2023; Davies *et al.*, 2025). The canary report belongs with the experiment verdict so promotion is traceable to evidence (Chen, 2026; Vellaisamy *et al.*, 2026).

1.3.4 Rollback drills

A rollback path that has never been exercised is a hope, not a mechanism (Chen, 2026, 2020; Davies *et al.*, 2025). The tuning workflow should include a scheduled rollback drill that restores the previous artifact under simulated load and measures

the recovery time (Vellaisamy *et al.*, 2026; Chen, 2026). The drill verifies that the fingerprinted human-prior schedule is available, that the deploy step is one command, and that telemetry confirms the switch (Chen, 2020; Davies *et al.*, 2025). Rollback drills also expose dependencies that were forgotten: a candidate that changed a shared runtime component cannot be rolled back by swapping only its kernels (Chen, 2026; Vellaisamy *et al.*, 2026). Teams that drill rollback treat incidents as rehearsed operations; teams that skip drills improvise under load (Davies *et al.*, 2025; Chen, 2026).

1.3.5 Feedback cadence and re-tuning triggers

The loop does not end at merge; it returns to profiling on a cadence and on explicit triggers (Chen, 2020, 2026; Vellaisamy *et al.*, 2026). Nightly dashboards compare the shipped counters against the hypothesis-card expectations and alert on drift (Davies *et al.*, 2025; Chen, 2026). Explicit triggers—driver upgrades, compiler upgrades, model releases, new context buckets, new SKUs—reopen the classification matrix before any tuning starts (Vellaisamy *et al.*, 2026; Chen, 2020). A monthly review should walk the highest-impact drift and decide whether it is environment, workload, or artifact movement (Davies *et al.*, 2025; Chen, 2026). This cadence is what keeps the workflow alive between projects and prevents it from fossilizing into a document nobody reads (Chen, 2020; Vellaisamy *et al.*, 2026).

1.3.6 Escape hatches and incident response

The tuning workflow should define what happens when a candidate that passed every gate still fails in production, because that moment is when trust in the system is made or broken (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). The incident response starts by switching traffic to the fingerprinted prior artifact, then preserves the failing artifact and its reports for diagnosis (Chen, 2020, 2026). The postmortem compares the failing cell against the canary cell to find the difference—traffic mix, bucket, driver, or environment (Davies *et al.*, 2025; Kwon *et al.*, 2023). The outcome should update the workflow: a new gate, a wider canary window, or a new classification cell (Chen, 2020; Vellaisamy *et al.*, 2026). Teams that treat escape-hatch incidents as workflow failures improve the system; teams that blame the engineer repeat the incident (Chen, 2026; Davies *et al.*, 2025).

1.3.7 Cross-backend promotion cards

Promoting a change from one backend to another requires a new hypothesis card, not a copy of the old one, because the counters and knobs differ (Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A Triton fusion win on GPU cannot be assumed to survive an XLA or IREE compile on the same model; the mechanism must be re-derived for the new backend's passes (IREE / iree-org contributors, 2024; Chen *et al.*, 2018; Chen, 2026). The promotion card should state which artifact moved, which backend it came from, and which new artifact and bucket set will validate it (Chen, 2020; Davies *et al.*, 2025). Cross-backend promotion without a new card is how teams inherit someone else's illegal schedule (Chen, 2026; Vellaisamy *et al.*, 2026). Each promotion that passes becomes a row in the classification matrix, extending the workflow's evidence base (Davies *et al.*, 2025; Chen, 2026).

1.3.8 Artifact generation inside the workflow

Every experiment should produce the artifact set its gate will consume: the compiled binary, its fingerprint, the compile report, the IR diff, and the benchmark rows (Chen, 2020, 2026; Vellaisamy *et al.*, 2026). Artifact generation should be scripted in the same build system that release uses, so the experiment artifact is a subset of the release artifact rather than a one-off build (Davies *et al.*, 2025; Chen, 2026). A candidate whose artifact cannot be regenerated is not a candidate; it is an anecdote (Chen, 2020; Vellaisamy *et al.*, 2026). The workflow should therefore fail fast when artifact generation diverges from the release path, because that divergence is where benchmarked binaries and shipped binaries stop being the same thing (Davies *et al.*, 2025; Chen, 2026).

1.4 regression gates

Regression gates encode the workflow into CI so human discipline survives headcount churn (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Minimum bar for merge:

- Decode-cell A/B on agreed buckets passes on bytes/token, launches/token, ms/token (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).
- No new global allocations in captured graph regions (GPU) or new DDR

hops (NPU) (NVIDIA, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

- IR diff attached; roofline or buffer accounting for target SKU (SemiAnalysis, 2024; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2022; Chen and YiRage Contributors, 2026; AMD, 2024a; Kwon *et al.*, 2023).
- Fingerprint or schedule artifact versioned with compiler git SHA (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Nightly jobs replay the full cell matrix across representative SKUs; drift alerts fire when counters move more than agreed ϵ without an approved compiler bump (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gates fail closed: ambiguous profiles block merge until reproduced (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 21 benchmarks supply the harness; Chapter 25 automates search inside the same gates (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Production runbooks (Chapter 26) assume these gates ran before containers ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Flaky gates erode trust: quarantine tests with high variance until environment drift is fixed—do not weaken thresholds to greenwash CI (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Pin driver, CUDA, and compiler versions in gate metadata; unexplained drift triggers infra tickets before compiler blame (NVIDIA, 2022, 2024; Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; AMD, 2024b).

Security and determinism gates belong beside performance: numerics diffs beyond agreed tolerance block merge even when ms/token improves (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Regression gates protect users, not just leaderboard scores (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Publish gate failures publicly inside the compiler team: failed experiments teach more than silent reverts (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). A culture that hides negative results repeats the same illegal fusion twice (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

1.4.1 Gate hierarchy

Regression gates should be layered so each layer is cheap and catches the failures the previous layer cannot (Chen, 2020, 2026; Vellaisamy *et al.*, 2026). The compile layer rejects artifacts that fail to build, fail legality checks, or introduce new allocations in captured regions (NVIDIA, 2024; AMD, 2024a). The cell layer replays the decode-cell benchmark on the agreed buckets and rejects counter regressions (Davies *et al.*, 2025; Chen, 2026). The integration layer runs the artifact through the serving harness and checks numerical and behavioral parity (Chen, 2020; Davies *et al.*, 2025). The canary layer is the final gate that confirms the artifact under real traffic (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Each layer publishes its report so a failure at any level points to the exact artifact and measurement (Chen, 2026; Davies *et al.*, 2025). Layering keeps CI fast because most candidates die at the cheap compile layer (Chen, 2020; Vellaisamy *et al.*, 2026).

1.4.2 Tolerance derivation

Gate tolerances should be derived from measurement noise, not chosen by negotiation (Davies *et al.*, 2025; Chen, 2026, 2020). Run the same artifact repeatedly on the target hardware, measure the counter distributions, and set epsilon from the observed variance plus a safety margin (Vellaisamy *et al.*, 2026; Chen, 2026). A tolerance tighter than the hardware noise will produce flaky gates; a tolerance looser than the product SLO will let regressions ship (Chen, 2020; Davies *et al.*, 2025). The derivation should be re-run when hardware, driver, or workload changes, because noise characteristics change with all three (Chen, 2026; Vellaisamy *et al.*, 2026). Publishing the derivation with the gate makes tolerance decisions reviewable instead of mysterious (Davies *et al.*, 2025; Chen, 2026).

1.4.3 Quarantining flaky gates

A flaky gate is a process failure that should be quarantined, diagnosed, and re-enabled, never silently weakened (Chen, 2026, 2020; Davies *et al.*, 2025). Quarantine means the gate stops blocking merges while an owner investigates the variance source—usually environment drift, shared machine contention, or an inadequate tolerance (Vellaisamy *et al.*, 2026; Chen, 2026). The quarantine should be visible in the dashboard with a ticket and an owner, so it cannot become a permanent bypass (Chen, 2020; Davies *et al.*, 2025). Re-enabling requires the diagnosis and the tolerance derivation update (Chen, 2026; Vellaisamy *et al.*, 2026). Teams that weaken thresholds to greenwash CI trade short-term green for long-term blindness (Davies *et al.*, 2025; Chen, 2026).

1.4.4 Numerics and determinism gates

Performance gates must be paired with numerical gates because a faster artifact that changes outputs is a defect (Chen, 2020; Davies *et al.*, 2025; Chen, 2026). Each candidate should be compared against the reference under a bounded-error metric at the served buckets, with the tolerance derived from the model's quality requirements (Dao *et al.*, 2022; Chen, 2026). Deterministic mode should lock reduction order where the product needs bitwise reproducibility, while interactive serving may allow bounded drift if it is measured and logged (Chen, 2020; Davies *et al.*, 2025). Numerical gates belong beside performance gates in the same CI run, because separating them lets a fast-but-wrong candidate pass review twice (Chen, 2026; Davies *et al.*, 2025).

1.4.5 Cross-hardware gate matrix

Multi-target fleets need a gate matrix so a change that helps GPU but hurts CPU or NPU cannot merge silently (Chen, 2020; AMD, 2024a; Chen, 2026). The matrix rows are the device classes and context buckets the change affects; the columns are the compile, cell, integration, and numerical gates (Davies *et al.*, 2025; AMD, 2024b; Chen, 2026). A candidate that only runs on GPU should be labeled GPU-only and blocked from heterogeneous promotion, not given a pass because its GPU numbers improved (Chen, 2020; Vellaisamy *et al.*, 2026). The matrix cells should be filled by the per-target classification evidence from the profiling station (Chen, 2026; Davies *et al.*, 2025). When a fleet adds a target, the matrix must be extended and the prior artifacts re-gated before the rollout (AMD, 2024b; Vellaisamy *et al.*, 2026). A gate matrix is how the workflow enforces the book's multi-hardware thesis at CI time (Davies *et al.*, 2025; Chen, 2026).

1.4.6 Operating documentation outputs

The workflow should produce living operating documents rather than a single tuning guide (Chen, 2020, 2026; Vellaisamy *et al.*, 2026). The decode-cell protocol, the classification matrix, the current hypothesis queue, and the gate dashboard are each maintained artifacts with owners and update cadences (Davies *et al.*, 2025; Chen, 2026). Release notes should link the artifact fingerprint, the gate reports, and the hypothesis cards that the release contains (Chen, 2020; Vellaisamy *et al.*, 2026). On-call runbooks should include the rollback drill and the incident template from this workflow (Chen, 2026; Davies *et al.*, 2025). Operating documentation turns the four stations into an institutional capability that survives individual engineers (Chen, 2020, 2026).

1.4.7 Gate failure learning review

Gate failures are learning data, and the workflow should hold a lightweight review of them on a cadence (Chen, 2020, 2026; Davies *et al.*, 2025). The review clusters failures by cause—misclassification, tolerance error, environment drift, or a genuine regression—and decides what the workflow should change (Vellaisamy *et al.*, 2026; Chen, 2026). A cluster of misclassifications means the classification training or the decision tree needs improvement; a cluster of environment failures means the protocol needs stronger fingerprints (Chen, 2020; Davies *et al.*, 2025). The review should publish its conclusions next to the gate dashboards so the loop visibly improves (Chen, 2026; Vellaisamy *et al.*, 2026). Without a learning review, the

gate system punishes failures without capturing their lessons (Davies *et al.*, 2025; Chen, 2026).

1.4.8 Gate report cadence

The gate system should publish reports on a cadence so the whole team can watch the workflow's health (Chen, 2020, 2026; Vellaisamy *et al.*, 2026). The report includes pass rates per gate layer, quarantine list, tolerance revisions, and the counter drift dashboard (Davies *et al.*, 2025; Chen, 2026). A weekly digest that summarizes these numbers keeps the workflow visible without requiring every engineer to query the CI system (Chen, 2020; Davies *et al.*, 2025). When pass rates drop, the digest should point to the dominant failure cluster and the open diagnosis ticket (Chen, 2026; Vellaisamy *et al.*, 2026). A visible gate report is the difference between a process the team trusts and a process the team discovers after an incident (Davies *et al.*, 2025; Chen, 2026).

1.4.9 SLO-to-counter mapping

Product SLOs must be translated into counter targets before the workflow can gate on them (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). An ms/token SLO decomposes into a launch budget and a bytes budget at the served buckets, and those budgets become the hypothesis-card acceptance rules (Chen, 2020; Davies *et al.*, 2025). The mapping should be explicit and reviewed when the SLO changes, because a looser SLO may end tuning early while a tighter one reopens the classification matrix (Chen, 2026; Vellaisamy *et al.*, 2026). Teams that gate only on ms/token cannot tell which lever moved; teams that gate only on counters lose the product meaning (Davies *et al.*, 2025; Chen, 2026). The mapping document connects the workflow's measurements to the product's promises (Chen, 2020; Vellaisamy *et al.*, 2026).

1.5 Key Takeaways

- **Profile the production shape, not a proxy.** Hardware-aware profiling fails the moment a team measures prefill shapes, batch-32 training graphs, or synthetic GEMMs while production runs batch-1 decode with paged KV (Kwon *et al.*, 2023; Davies *et al.*, 2025). Freeze decode cells and normalize GPU, CPU, and NPU tools to the same bytes/token and launches/token so the numbers are comparable across targets (Chen, 2026; Vellaisamy *et al.*,

2026).

- **Classify the bottleneck before touching a knob.** Bottleneck locating—deciding whether a workload is launch-, memory-, residency-, or framework-bound—is what separates senior compiler engineers from kernel tourists, because changing a compiler flag before classifying wastes effort on the wrong term (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The production surprise is usually orchestration or residency, not the matmul the tourist reaches for first (NVIDIA, 2024; Vellaisamy *et al.*, 2026).

- **Change one knob per experiment against a human baseline.** A repeatable tuning workflow turns classifications into an experiment queue: establish a human-prior baseline, vary one knob at a time, and canary on the fleet before promotion (Chen, 2020; Chen and YiRage Contributors, 2026). Bundling changes makes a regression impossible to attribute, which is why the discipline is one variable per measurement (Davies *et al.*, 2025).

- **Encode the workflow as regression gates.** Human discipline does not survive headcount churn, so the workflow must live in CI: enforce counter non-regression, residency legality, and versioned, fingerprinted artifacts on every merge (Chen, 2020; Chen and YiRage Contributors, 2026). A gate that checks bytes/token and launches/token at the decode cell is what stops a microbench-only win from shipping (Chen, 2026; Vellaisamy *et al.*, 2026).

- **Profile-first beats tile-size folklore.** The recurring lesson of end-to-end tuning is that measured decode counters, not inherited tile-size heuristics, decide what to change (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Folklore tuned on someone else's prefill workload is exactly the trap a frozen decode cell and a profile-first loop are designed to avoid (SemiAnalysis, 2024).

Practical decision rules. Six rules compress this chapter for a working session. First, never tune before the frozen decode cell and classification label exist, because unclassified tuning is gambling (Davies *et al.*, 2025; Chen, 2026). Second, change one knob per experiment and write the expected counter movement on the card before building (Chen, 2020; Vellaisamy *et al.*, 2026). Third, keep the human-prior schedule fingerprinted and reachable so every candidate competes against a known baseline (Chen, 2026; Davies *et al.*, 2025). Fourth, gate every merge on the layered CI plus canary, with tolerances derived from measured noise (Chen, 2020, 2026). Fifth, quarantine flaky gates with owners instead of weakening them, and publish negative results alongside wins (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Sixth,

re-open the classification matrix on every driver, compiler, model, bucket, and SKU change rather than assuming the last labels still hold (Chen, 2026; Davies *et al.*, 2025). Teams that internalize these rules turn tuning into a system; teams that skip them repeat the same misclassification every quarter (Vellaisamy *et al.*, 2026; Chen, 2026).

1.6 Conclusion

Compiler-driven end-to-end performance tuning is the operational glue between backends (Part VI), benchmarks (Chapter 21), automated search (Chapter 25), and production packaging (Chapter 26) (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Without hardware-aware profiling and regression gates, compiler teams optimize kernels while decode goodput flatlines (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gate merges on buffer accounting and reproducible decode cells—never on a single Nsight screenshot (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). This profile-first, regression-gated workflow is general, but it pays off most on the hardest decode targets, which is where the next chapter applies it: Chapter **??** specializes the workflow for LLM and MoE compilation, where data-dependent expert routing and growing KV caches stress exactly the launches/token and bytes/token counters this chapter taught teams to measure (Liu *et al.*, 2024; Kwon *et al.*, 2023).

Bibliography

AMD (2024a). Ai engine technology. https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html. 1, 2, 3, 4, 6, 9, 10, 13, 14, 18, 19, 21, 24

AMD (2024b). AMD XDNA architecture. https://www.AMD.com/en/technologies/XDNA.html. 1, 2, 3, 4, 5, 6, 9, 10, 13, 14, 18, 19, 21, 24

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018). TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. Graph- and operator-level optimization, operator fusion, memory latency hiding, ML-based cost model; CPU/GPU/FPGA. 17

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. https://github.com/chenxingqiang/YiRage. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 9, 10, 13, 14, 17, 18, 19, 23, 24

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 2, 3, 4, 5, 9, 10, 13, 14, 18, 19, 24

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 9, 10, 12, 13, 14, 18, 19, 20, 24

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 13, 14, 17, 18, 19, 23, 24

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. `https://iree.dev`. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; `.vmfb` fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 17

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 18, 19, 22, 24

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 2, 3, 4, 5, 9, 10, 13, 14, 18, 19, 24

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 3, 4, 5, 9, 10, 11, 13, 14, 18, 19, 24

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 3, 4, 5, 7, 9, 10, 12, 13, 14, 18, 19, 23, 24

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 3, 4, 5, 9, 10, 13, 14, 18, 19, 23, 24

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24